Programmiersprachen

Aspekte einer Sprache:

- **Syntax** (richtiger Aufbau von Zeichenketten)
  - Alphabet & Grammatik bilden die Syntax einer Sprache
- **Semantik** (Bedeutung von Zeichenketten)
  - syntaktisch richtige Zeichenketten
- **Pragmatik** (persönliche Interpretation)
  - sauberen, effizienten Code

- Alphabet: Menge der zulässigen Zeichen, die benutzt werden dürfen
- Grammatik: Menge aller Vorschriften
  - Regeln, die angeben auf welche Weise sich Zeichen/-reihen zu kombinieren sind
  - Besteht aus:
    - G = (N, T, P, S)
- **Teminalsymbole** T
  - Zeichenvorrat/Menge Grundsymbole, aus denen Worte gebildet werden
- **Nonterminalsymbolen** N
  - Alphabet der Hilfssymbole, Variablen / metasprachlichen Größen
  - Platzhalter für diverse Sprachelemente
- **Startvariable** S (unter Nonterminalsymbolen)
- **Produktionsregeln** P / Ersetzungsregeln
  - Legen fest, wie gewisse Symbole durch Hilfs- / Terminalsymbole ersetzt werden können

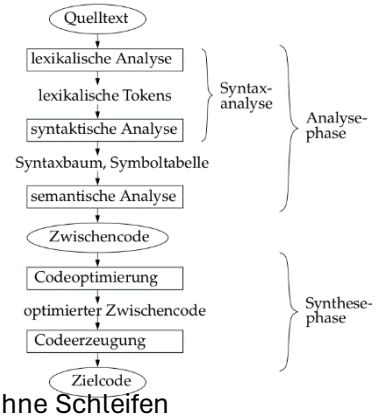
Allgemeine Backus-Naur-Form (ABNF)

- Darstellungsform für die formale Definition der Syntax von Programmiersprachen
- Folgende (metasprachliche) Symbole
  - ::=
    - Definitionszeichen/Ersetzungszeichen
  - |
    - Entweder-oder-Zeichen
  - < & >
    - Hilfsmittel zur Kennzeichnung der Variablen, kann auch fett markiert sein
  - ⇒
    - Ableitungszeichen
  - { und }
    - Eingeschlossene Einheiten bei Bedarf in einem Ableitungsschritt beliebig oft (keinmal/1x/mehrmals) hintereinander angeschrieben
  - [und]
    - Eingeschlossene Einheit kann bei Ableitung höchstens 1x (keinmal/1x) verwendet werden

Chomsky Hierarchie

- Modell zur Kategorisierung & Unterscheidung von vers. Arten der Sprachverarbeitung
  - Type 0: Phasenstrukturgrammatik
    - keine Einschränkungen
  - Type 1: Kontextsensitive Grammatik
    - + Längenbeschränkung
  - Type 2: Kontextfreie Grammatik
    - Linke Seite: ein nichtterminales Symbol
    - Rechte Seite: beliebige Kombi aus nicht- & terminalen Symbolen
  - Type 3: Reguläre Grammatik
    - Rechte Seite: Genau 1 Terminalsymbol & optional 1 Nichtterminalsymbol

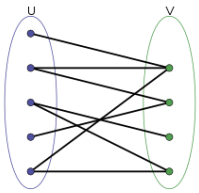
Compiler



- Quelltext
- lexikalische Analyse (Typ-3 Sprache)
- lexikalische Tokens
- syntaktische Analyse (Typ-2 Sprache- Syntaxbaum)

Graphentheorie

- einfacher Graph (schlichter Graph)
  - ungerichteter Graph ohne mehrfachkanten & ohne Schleifen
  - geordnetes Paar (V, E)
    - V … endliche Menge von Knoten
    - E … Menge von Kanten
  - Beispiel: Nachbarschaftsbeziehungen zwischen Deutschland & Nachbarländern
- Schlinge (Schleife)
  - Kante mit dem gleichen Anfangs- & Endknoten
- Mehrfachkanten/Multikanten
  - Zwischen 2 Knoten verlaufen in einem Multigraphen mehrere gleichartige Kanten
  - Einzelne Kanten = parallele Kanten
- Komplement
  - Spezieller Graph, den man aus einem gegebenen Graph erhält
    - Gleiche Knoten wie Ursprungsgraph
    - unterscheidet sich aber in Kanten (genau die Kanten, die der Ursprungsgraph hat)
- Clique
  - Jeder kennt jeden
- Anticlique
  - Keiner ist mit irgendjemanden verbunden (Spanien, Polen, Griechenland)
- Vollständige Graphen
  - Einfachen Graphen, jeder Knoten mit jedem anderen durch 1 Kante verbunden
- (Sub-)Graph
  - Beziehung zwischen 2 Graphen
- Bipartit Graphen (Zuordnungsprobleme)
  - Alle Knoten werden aufgeteilt in 2 Mengen
  - Verbindungen sind nur zwischen Knoten der anderen Menge & nicht innerhalb derselben Menge



- Chromatische Zahl (Färbeproblem)
  - Beispiel Europakarte: Wie viele Farben braucht man, um alle Knoten darzustellen, sodass alle Nachbarn unterschiedliche Farben haben
  - Sobald ebene Fläche nie < 4 Farben
  - Exklaven (2 Länder gehören zusammen=gleiche Farbe, aber woanders): 5 Farben
    - Anticliquen bilden
    - Partition: Zerlegung/Klasseneinteilung
- Matrix Darstellungsform (moderne Graphen werden in DB gespeichert)
  - Isomorph Graph: strukturell gleiche Graphen
- Grad eines Graphen
  - Anzahl von Kanten, die an Knoten angrenzen
- Weg (manchmal auch als Pfad bezeichnet)
  - Verbindung von Start- zu einem Endknoten
  - Folge von Knoten, in welcher jeweils 2 aufeinanderfolgende Knoten durch Kante verbunden
- Kreis: jeder Knoten darf nur einmal vorkommen bis auf der Startknoten
  - Geschlossener Pfad
- Zyklen
  - Geschlossener Weg
- Euler Kreis
  - Geschlossener Kantenzug, der jede Kante aus dem Kreis genau einmal benutzt
- Abstand, zentraler Knoten & Radius
  - Abstand: Summe der Distanzen/Gewichte, um von Knoten A nach B zu kommen
  - Zentraler Knoten: Knoten, der die wenigste Distanz zu allen anderen Knoten hat
  - Radius: Abstand, vom zentralen Knoten zum entferntesten Knoten
- Zusammenhang
  - Komm ich von A nach B?
- Separator (Netzwerktechnik) = trennende Knotenmenge (Knoten wird entfernt)
  - Bei Ausfall (Router)
  - Graph spaltet sich in 2 Teilgraphen
- Schnitt = trennende Kantenmenge (Knoten und Kanten werden entfernt)
  - Verbindung wird entfernt
  - Graph spaltet sich in 2 Teilgraphen
- Aufspannende Bäume (Dijkstra)
  - Zusammenhängender Graph = Baum
- Planare Graphen (Maps)
  - Graphen kreuzungsfrei darstellen
- Gewichteter Graph
  - Nicht mehr mit Kanten, sondern z.B. Distanz
- Gerichteter Graph
  - Senke: Knoten, der keinen Ausgang hat (nur Eingänge)
  - Quelle: Knoten, der nur Ausgänge hat

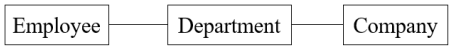
Software-Pattern

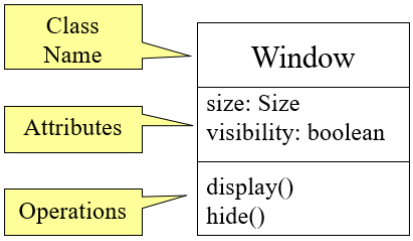
- Allgemein: Was sind Software-Pattern?
  - Typische Lösungen für häufig auftretende Probleme im Softwaredesign
- Factory-Pattern (z.B. Programm zum Erzeugen von Formen)
  - Familien von verwandten Objekten produzieren, ohne konkrete Klassen

- Observer (Verhaltensdesign-Muster)
  - mit dem man Abonnement-Mechanismus definieren kann, um mehrere Objekte über Ereignisse zu informieren, die Objekt passieren, das man beobachtet
- Composite (Child, ...)
  - Objekte zu Baumstrukturen zusammenfügen
- Iterator (Verhaltensdesign-Muster)
  - Mit dem man Elemente einer Sammlung durchqueren kann, ohne zugrunde liegende Darstellung (Liste, Staple, Baum usw.) freizusetzen
- Singelton => für Globaler Zugriff, Ressourcenverwaltung, Statusverwaltung
  - Sicherstellen, dass Klasse nur eine Instanz hat

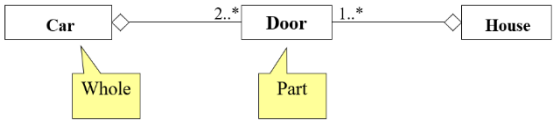
UML

- Indirekte Verbindungen
  - Employee auch mit Company verbunden

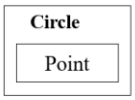




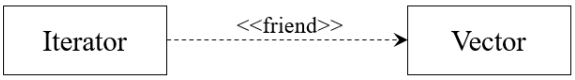
- Assoziation
  - Multiplizität: aus Sicht der verbundenen Entität
  - Reflexive Beziehung (4.Klasse muss für 5.Klasse)
- Aggregation
  - Teil-Ganzes-Beziehung



- Komposition
  - Essenzielles Teil (Es gibt keinen Kreis, der keinen Punkt hat)
- Generalisation (Vererbung)
  - Von Sub-Klasse => Super-Klasse
  - Sub-Klasse kennt Super-Klasse



- Abhängigkeit (Dependency)
  - Kein Teil von aber abhängig (ohne geht auch nicht) z.B. Kind, Eltern
- Relization
  - Pfeil zum Interface



Zusätzliche Fragen

- Was wird bei unserem NW übertragen?
  - Netzwerkklasse serialisiert
    - Messageklasse: was wird in der MSG-Klasse gemacht => XML Seri
  - Problem Binär-Datenübertragung
    - Man hat keinen Endtag und den braucht man (länge übergeben, um Problem zu lösen)
  - TCP-Client
    - Programm um über TCP/IP Verbindung zum Client aufgenommen wird
- Threads
  - Warum in dem Zusammenhang Threads?
  - Wie viele brauche ich? Server? Client?
    - Client: 1 Thread (1 Main und 1 zusätzlichen pro Client zur Connection)
    - Server: 1 Threat pro Client + Main Thread

POS Matura Theorie Zusammenfassung

- XML: Welche Regeln muss man beachten?
  - Standard Konstruktor public
  - Jedes Element muss Serilialisierbar ist (außer XMLIgnore)
  - Multitype Serialisierung => Polymorphie Problem: mittels Wrappers lösbar
    - Beispiel: Expression
      - Operator
      - Number: andere Properties als Operator
- Was ist LINQ? Für was eingesetzt? Language Integrated Query
  - Abfragesprache
  - SQL ähnlich
  - Linq2db = ORM-Mapper
- Was ist ORM?
  - Technik, die den Zugriff in einem Programm auf eine Datenbank ermöglicht
  - Objekte zu DB-Objekte übertragen (Entitys zu Objekten)
- Wo gibt's Problem mit ORM in Vererbung?
  - Wie speichert man die Daten wo in der DB ab?
- Wann brauche ich Databinding?
  - Dort, wo sich Daten ändern
  - Wenn man auf Dinge in der UI zugreifen will, die sich ändern können
- Wann braucht man Datenbindings nicht?
  - Sobald man statische Werte hat, da sie sich eh nicht ändern
- Wann funktionieren Datenbindings nicht?
  - Es gibt im Wesentlichen 3 verschiedene Verfahren, um Vererbungshierarchien auf Datenbanktabellen abzubilden
    - Tabelle pro Vererbungsstrategie, Unterklasse, konkrete Klasse
  - Wenn Bindings nicht funktionieren ist das ein Bug den man beheben muss
  - Fehlerquelle:
    - iNotifyChangedProperty fehlt
- Lambda und Lin
  - 1x SQL ähnliche selects
  - Lambda Funktionen (anonyme Funktionen)
- Parser
  - Wir bauen ihn von oben nach unten
  - Tokenizer: Typ 3 Sprache
  - Kein Tokenizer: Typ 2 Sprache